

Randers-UMAP on the Swiss Roll: Three Initialisation Strategies and the Ramp Mechanism

Progress Report, Week of August 7–12

Rümeysa Bilik

Max Planck Institute for Mathematics in the Sciences

Supervised by Prof. Jürgen Jost

August 12, 2026

1 Methodology recap

All three pipelines below share the same skeleton, so this section is intentionally short.

Distance matrix. `randers_bridge.py::compute_dist_matrix` turns (X, ω) into an asymmetric geodesic distance matrix D_{asym} : build a neighbourhood graph on X , perturb each edge weight by $\langle \omega_i, x_j - x_i \rangle$, run directed shortest-path. This week the neighbourhood-graph step was rewritten (§4) to match `lwileczek/isomap`’s threshold construction instead of `scikit-learn`’s k -NN graph.

Randers metric. Every method ultimately positions each node i at $y_i \in \mathbb{R}^2$ together with a drift vector b_i , $\|b_i\| \leq 1 - \delta$, so that

$$F(i \rightarrow j) = \|y_j - y_i\| + \langle b_i, \hat{e}_{ij} \rangle \quad (1)$$

approximates $D_{\text{asym}}[i, j]$. The three pipelines differ only in *how* Y and B are initialised and *what* update rule then refines Y – see §3.

Force-directed update (used by two of the three). `randers_umap.py::randers_umap_fit` lays Y out with a dense, full-batch UMAP-style attraction/repulsion graph layout, not gradient descent against a loss. D_{asym} is first turned into a fuzzy membership graph $\mu_{ij} \in [0, 1]$ exactly as in UMAP itself (Section 3.1 of the original paper): a per-point ρ_i, σ_i calibration on the k nearest neighbours so every node’s total soft-neighbourhood weight matches $\log_2 k$. The drift then enters the layout by replacing the ordinary embedding distance and force direction with

$$\rho_{i \rightarrow j} = \|y_j - y_i\| + b_i \cdot (y_j - y_i), \quad g_{ij} = \hat{e}_{ij} + b_i, \quad (2)$$

inside UMAP’s own attractive/repulsive coefficients $\text{attr}(\rho), \text{rep}(\rho)$ (the usual (a, b) curve fit to `min_dist/spread`). The net force on node i is

$$\text{force}_i = \sum_j \left[\mu_{ij} \text{attr}(\rho_{ij}) - (1 - \mu_{ij}) \text{rep}(\rho_{ij}) \right] g_{ij}, \quad (3)$$

and Y is updated by pure force accumulation, $Y \leftarrow Y + \eta \text{force}$, every epoch. Setting $b_i \equiv 0$ everywhere recovers vanilla UMAP exactly, since then $\rho = \|y_j - y_i\|$ and $g = \hat{e}_{ij}$.

Locating the drift via virtual points. B itself is not learned by this force loop – `B_fixed` is computed once, *before* training, and never updated afterwards (only its *magnitude* is phased

in, see the ramp mechanism, §2). The main construction (Isomap and SVD-RADAR pipelines, §3) locates it geometrically: for every real point x_i , place an imaginary target

$$x_i^{\text{virt}} := x_i + \omega_i \quad (4)$$

in the same ambient space – a genuine position, not an abstract offset, since ω is represented in the same coordinates as X . All $2n$ real+virtual points are then embedded *jointly* with plain, drift-off UMAP ($b \equiv 0$ in Eq. (2)) on their symmetric geodesic distance. The located drift is simply the displacement between the two embedded images of the same node,

$$B_{\text{located}} := y_i^{\text{virt}} - y_i^{\text{real}}, \quad b_i \leftarrow b_i \cdot \min\left(1, \frac{1-\delta}{\|b_i\|}\right), \quad (5)$$

clipped to the Randers bound. B_{located} is then frozen and passed as **B.fixed** to the real (asymmetric) embedding: it still shapes Y 's force every epoch through Eq. (2), but its own value never changes for the rest of training – it is rigidly “attached” to node i and rides along as y_i moves.

2 The ramp mechanism

2.1 What it is

`randers_umap_fit` accepts a frozen drift field **B.fixed** (computed once, attached to each node, never retrained – see §3). A scalar schedule $s(\tau)$ optionally controls how much of that frozen field is actually applied at epoch τ :

$$s(\tau) = \begin{cases} 0 & \tau/T \leq 0.30 \\ (\tau/T - 0.30)/0.40 & 0.30 < \tau/T < 0.70, \\ 1 & \tau/T \geq 0.70 \end{cases}, \quad B(\tau) = s(\tau) \cdot B_{\text{fixed}}, \quad (6)$$

T the total epoch count. **ramp=False** fixes $s \equiv 1$: the frozen field is applied at full strength from epoch 0. Only s changes with τ – B_{fixed} 's direction is never touched, so this is not a re-optimisation of B , only a magnitude schedule.

2.2 Why full strength from epoch 0 can fail

From (1), Cauchy–Schwarz bounds the drift term:

$$|b_i \cdot (y_j - y_i)| \leq \|b_i\| \cdot \|y_j - y_i\|. \quad (7)$$

So $\rho_{i \rightarrow j} \in [(1 - \|b_i\|)d_{ij}, (1 + \|b_i\|)d_{ij}]$. For a well-behaved node ($\|b_i\| \approx 0.05$, the typical case at $n = 300$, see Table 1) this is a gentle $\pm 5\%$ perturbation of distance. For a node sitting at the Randers clip boundary ($\|b_i\| \approx 0.99$) it is a $\pm 99\%$ swing: ρ can fall to $\approx 0.01 d_{ij}$ or rise to $\approx 1.99 d_{ij}$ for the *same* pair of points, depending only on which node is source. The (a, b) attraction/repulsion curve was calibrated (**min_dist=0.1**, **spread=1.0**) assuming ρ behaves like an ordinary, well-scaled UMAP distance – not one that can swing this violently. Applying that swing at full strength ($s = 1$) from epoch 0, while Y is still the compact, just-finished output of the locate step and has not yet found its natural scale, produces erratic per-edge forces for every saturated node: a minority of nodes get flung to extreme positions while the majority, left with only ordinary (and, at this n , comparatively weak dense-repulsion) attraction, collapse toward a shared centroid.

run	locate epochs	mean $\ b\ $	max $\ b\ $	clipped (of 2000)
$k = 20$, default locate (500 ep.)	500	0.053	0.990	11
$k = 12$, short locate (100 ep.)	100	0.143	0.990	206

Table 1: Locate-step convergence, not k , is the dominant driver of B_{located} ’s saturation – see the caveat in §4.

2.3 Empirical evidence, $n = 2000$

With **ramp=False** on the 206/2000-saturated run: apply-step epoch 0 already shows mean $|\rho - d| = 0.797$ (against 0.124 at $n = 300$, 0 saturated nodes), and the final embedding extent reaches 149.6 (against 19–24 at $n = 300$) – see Figure 1: a tight central clump with a handful of nodes flung to the edges. Switching only **ramp=True**, same data, same seed, same B_{located} : Figure 2 shows a clearly organised, colour-separated layout by epoch 20–40, because $s \approx 0$ during the first 30% of epochs lets Y spread under ordinary UMAP forces *before* the saturated nodes’ extreme drift is switched on.

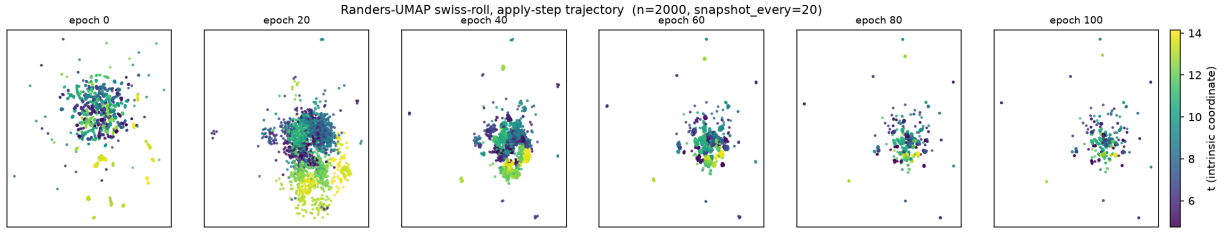


Figure 1: **ramp=False**, $n = 2000$: central collapse plus flung-out outliers.

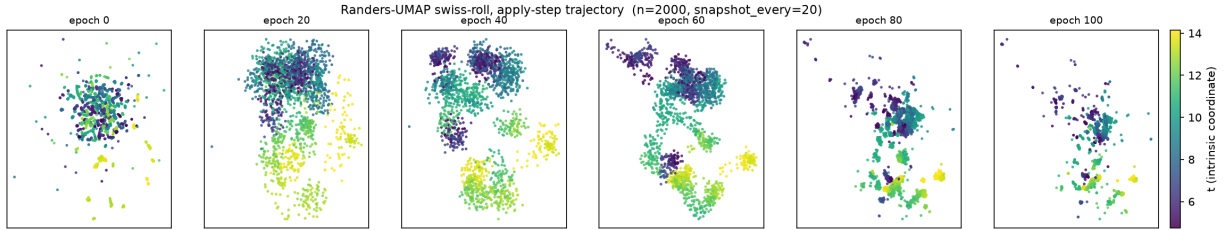


Figure 2: **ramp=True**, identical run otherwise: organised, colour-separated layout.

2.4 Caveat – not yet fully isolated

Table 1’s two runs differ in *both* k (20 vs. 12) *and* locate-step epoch count (500 vs. 100) simultaneously, because the short-locate-epoch run was originally set up for fast iteration, not as a controlled comparison. The saturation jump is more plausibly explained by the shorter, under-converged locate step than by k itself (see §4’s k discussion, which is a separate, independently-verified finding). A clean single-variable rerun ($k = 12$, locate-epochs=500) is on the list below (§5) but has not been run yet. **ramp=True** was adopted regardless because it makes the pipeline robust to a saturated B_{located} *whatever* its source, which is a reasonable thing to want independent of this open question. **ramp** is now a CLI flag (`--ramp`, default off) on all three scripts in §3, so it can be toggled without editing code.

3 Three initialisation strategies

All three now share the same force-directed update mechanism, Eq. (2). They differ only in how (Y_{init}, B) is obtained.

3.1 Isomap – `run_swiss_roll.py`

D_{asym} from `compute_dist_matrix` (the Isomap-style geodesic backbone). B and Y_{init} both come directly from the virtual-point locate step described in §1 (Eq. (5)): $B := B_{\text{located}}$, and $Y_{\text{init}} := y^{\text{real}}$ from that same joint embedding. No separate initialisation step beyond the locate step itself. Figure 3(a).

3.2 isumap – `run_swiss_roll_isumap.py`

D_{asym} comes from `distance_graph_generation.py` – isumap’s own, untouched library code (local ρ_i/σ_i normalisation, star-graph restriction, fuzzy t -conorm merge – see the file-by-file walkthrough discussed separately). This method has no notion of ω at all – its asymmetry comes purely from the directed k -NN structure isumap itself builds – so there is no virtual target to locate. B instead comes from a one-shot SVD of the skew-symmetric part, $B := \text{svd_init}(D_{\text{asym}}^\top - D_{\text{asym}})[:, :d]$, clipped to the Randers bound. Y_{init} is whatever `randers_umap_fit`’s own spectral initialisation gives on D_{asym} (no separate locate step here either). Figure 3(b).

3.3 SVD-RADAR – `run_swiss_roll_svd_radar.py`

Same D_{asym} as the Isomap pipeline (true ω). B is *still* the located construction above – unchanged, because B needs the true ω field to locate against, and SVD-of-Delta is a structurally different signal not yet validated as a substitute. The only thing that changes relative to §3.1 is Y_{init} : instead of the locate step’s own y^{real} , $Y_{\text{init}} := \text{svd_init}(D_{\text{asym}})[:, :d]$ (RADAR-style truncated SVD of the asymmetric matrix itself, no double-centering). This isolates a single variable against §3.1: does an SVD-derived starting position help or hurt, everything else held fixed? Figure 3(c). (This script previously used a completely different mechanism – SVD init feeding an Adam/Finsler-loss regression, `finsler_mds_freeB` – see §4.)

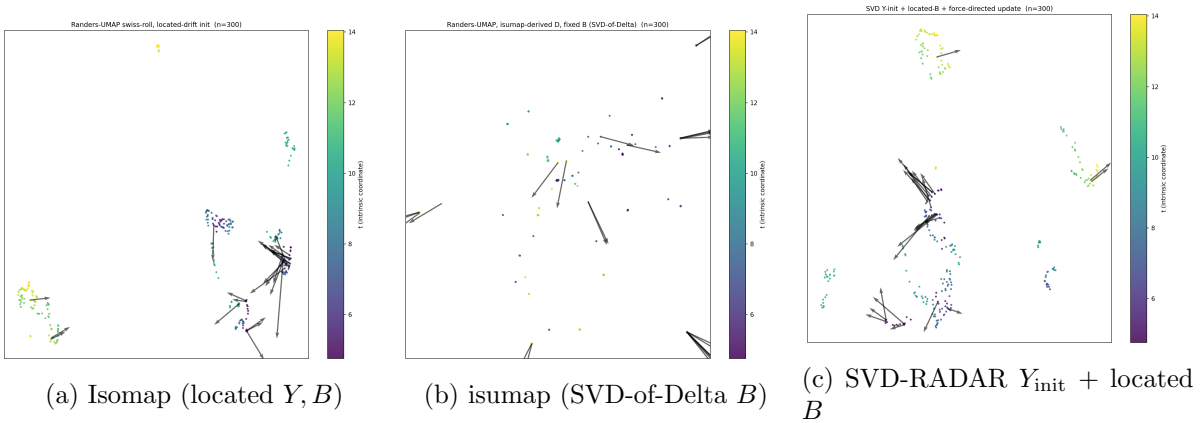


Figure 3: $n = 300$, 150 epochs, `ramp=False`, same seed/data across all three – for direct comparability rather than best-case tuning. All three still fragment into disconnected islands at this epoch count; see §5.

4 This week's changes

1. **Geodesic backbone rewritten to match `lwileczek/isomap`.** `compute_dist_matrix` previously built its neighbourhood graph via scikit-learn's `kneighbors_graph` (exactly k neighbours per point, adaptive to local density). It now uses a single global distance threshold ε (`dist <  $\varepsilon$   $\Rightarrow$  edge`), matching `lwileczek/isomap`'s `make_adjacency` at the advisor's request. ε is auto-derived from k so existing call sites are unaffected: $\varepsilon(k) = \max_i d_k(i)$, the largest per-point k -th-nearest-neighbour distance in the dataset.
2. **Diagnosed and fixed geodesic-backbone contamination.** Because $\varepsilon(k)$ is a single *global* value (set by the sparsest point), points in denser regions can end up connected to ambient neighbours across different windings of the roll – a classic Isomap short-circuit. Measured directly: at $n = 300$, $k = 20$, the true geodesic 20-NN of some points reach as far as $|t_i - t_j| = 6.87$ (75% of the entire t -range) – structurally wrong. Reducing k (20 $\rightarrow$ 12) is provably safe against this specific failure mode, since ε is monotone non-decreasing in k : at $n = 2000$, $k = 12$, the same statistic drops to max 0.58, median 0.11 – clean. (This is independent of the ramp finding in §2, which turned out to hinge more on locate-step convergence than on k – see the caveat there.)
3. **snapshot_every added to both update mechanisms.** Both `randers_umap_fit` and `finsler_mds_freeB` now record (Y, B) every N epochs (epoch 0 = true pre-training init, final epoch always included even under early stopping), enabling the trajectory grids used throughout this report and Figures 1–2.
4. **ramp exposed as a CLI flag everywhere.** `run_swiss_roll.py`, `run_swiss_roll_isomap.py`, and `run_swiss_roll_svd_radar.py` all take `--ramp` now (default off, matching the "locate once, freeze, attach at full strength" design intent) – previously `run_swiss_roll_isomap.py` silently used `randers_umap_fit`'s own `ramp=True` default without exposing it, an inconsistency caught while reviewing the three scripts side by side.
5. **run_swiss_roll_svd_radar.py redesigned.** Previously: SVD init feeding `finsler_mds_joint.py::finsler_mds_freeB`, an Adam optimiser minimising the Finsler reconstruction loss $\sum (F(i \rightarrow j) - D_{\text{asym}}[i, j])^2$ – a fundamentally different mechanism (gradient descent against a target matrix) from the other two scripts' force-directed layout. Rewritten to use `randers_umap_fit` instead, per §3, so all three scripts are now directly comparable on the same update rule. `finsler_mds_joint.py` itself is untouched and still importable for anyone who wants the Adam/loss variant.

5 Open items

1. **Isolate k vs. locate-epochs in Table 1.** Rerun $k = 12$ with the full 500-epoch locate step (not 100) to separate the two confounded variables cleanly.
2. **Figure 3 still fragments at $n = 300$, 150 epochs.** This is the same dense-force-at-scale limitation flagged in the previous located-drift report (`vanilla_n300.png`, drift completely off, still fragments) – not new, not fixed this week. A longer run (300+ epochs, per the direction-accuracy plateau found earlier) and/or a genuine stochastic-negative-sampling implementation (replacing the current dense all-pairs approximation) remain the two live candidates.
3. **Direction-accuracy comparison across all three methods.** Every image in this report is judged visually; a quantitative `direction_accuracy_swiss`-style comparison (recovered

B vs. true ω , projected into each embedding) across Isomap / isomap / SVD-RADAR has not yet been run this week.

4. **Server-scale** ($n \geq 1000$) **reruns of Figure 3.** All three-way comparisons here are at $n = 300$ for turnaround speed; the $n = 2000$ regime (where the ramp finding actually matters) has only been tested for the Isomap script, not yet isomap or SVD-RADAR.

Code

file	this week's role
<code>randers_bridge.py</code>	rewritten: eps-threshold backbone (item 1–2)
<code>randers_umap.py</code>	<code>snapshot_every</code> , <code>ramp</code> (already existed, now exposed)
<code>run_swiss_roll.py</code>	<code>--ramp</code> , <code>--snapshot-every</code> CLI flags added
<code>run_swiss_roll_isomap.py</code>	<code>--ramp</code> CLI flag added (was silently on)
<code>run_swiss_roll_svd_radar.py</code>	rewritten: force-directed update, SVD Y_{init} + located B
<code>finsler_mds_joint.py</code>	<code>snapshot_every</code> semantics aligned; no longer wired into <code>svd_radar</code> script